Team Members: Mike Jones, Cole Bennet, Michael Campbell

Dr.Fareena Saqib

ECGR 3183-091

Submitted on: 12/6/2024

**Computer Organization Final Project Report**

# Intro

This report will go into detail about the methods used, and knowledge learned throughout the semester to be able to implement and successfully output different components needed in a basic processor. These include but are not limited to, Program counter, Register File, Instruction memory, Data memory, and the Arithmetic logical unit.

# ISA

The instruction that was given was the ability to move data from register to register, compare two registers, and basic arithmetic such as addition, and substation. Register shifting left and right, could also be implemented to form multiplication and division. Condition branching, for simplicity we implement both if zero and not. The ability to combine or and two registers, while also having the ability to loop the instructions listed before. Else-or of two resisters was also required. Each of these required its own components and data paths for example the arithmetic logical unit required the following components, program counter, instruction memory, ALU, and a register memory unit. The program counter manages the address of the next instruction and increments it on every clock cycle. The instruction memory unit is responsible for receiving the instruction corresponding to the PC value. The Control unit decodes the instruction and makes a control signal to operate the other components such as muxes. The control unit intakes the instruction operation code and generates a signal to guide the other components to function, needed for the ALU and the data memory. The register file contains 32 general-purpose 32-bit registers, allowing data retrieval. The ALU computes the arithmetic and logical operations of the control unit. The data memory is used for the load and store instructions or the LDUR and STUR respectively.

# Implementation

The creation of each instruction that was desired for the processor to be able to compute was constructed separately from the entire code. This was done to ensure that any mistakes were limited to syntax and not logical errors due to the new code that was made. To also aid in this process we as a group made physical diagrams of the data and control path. For simpler instructions this was not needed as there was no logical confusion, however as the instructions started to overlap with the control, the diagrams helped give us a good understanding of how and what component would be used.  To implement these functions for the processor we initially implemented an easier processor function such as simply the PC and the instructions. Once this process was completed, then it was possible to continue to layer in the more complicated instructions and functions.

# Component Designs

The components that were included are the program counter, register file unit, instruction memory, data memory, and the arithmetic logical unit. These are also basic components of any processor to function fully. Designs are as follows: (if it is not appearing please click on the drop-down menu “VHDL CODE”)

## VHDL CODE: Data Memory

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity DataMemory is

  Port ( clk : in STD_LOGIC;

      address : in STD_LOGIC_VECTOR(31 downto 0);

      write_data : in STD_LOGIC_VECTOR(31 downto 0);

      mem_write : in STD_LOGIC;

      mem_read : in STD_LOGIC;

      read_data : out STD_LOGIC_VECTOR(31 downto 0));

end DataMemory;

architecture Behavioral of DataMemory is
```

```
   type memory_type is array (0 to 255) of STD_LOGIC_VECTOR(31 downto 0);
   signal memory : memory_type := (others => (others => '0'));
begin
   read_data <= memory(to_integer(unsigned(address(7 downto 0)))) when mem_read = '1' else
(others => '0');
process(clk)
begin
   if rising_edge(clk) then
      if mem_write = '1' then
         memory(to_integer(unsigned(address(7 downto 0)))) <= write_data;
      end if;
   end if;
end process;
end Behavioral;
```

# ALU

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;  -- Include numeric_std for arithmetic operations
entity ALU is
   Port ( A : in STD_LOGIC_VECTOR(31 downto 0);
        B : in STD_LOGIC_VECTOR(31 downto 0);
        ALUOp : in STD_LOGIC_VECTOR(3 downto 0);
        Result : out STD_LOGIC_VECTOR(31 downto 0);
        Zero : out STD_LOGIC);
end ALU;
architecture Behavioral of ALU is
begin
   process(A, B, ALUOp)
```

```
  variable A_unsigned : unsigned(31 downto 0);
  variable B_unsigned : unsigned(31 downto 0);
  variable result_unsigned : unsigned(31 downto 0);
begin
  -- Convert inputs to unsigned for operations
  A_unsigned := unsigned(A);
  B_unsigned := unsigned(B);
  case ALUOp is
    when "0000" =>  -- ADD
      result_unsigned := A_unsigned + B_unsigned;
    when "0001" =>  -- SUB
      result_unsigned := A_unsigned - B_unsigned;
    when "0010" =>  -- AND
      result_unsigned := A_unsigned and B_unsigned;
    when "0011" =>  -- OR
      result_unsigned := A_unsigned or B_unsigned;
    when "0100" =>  -- LSL
      result_unsigned := A_unsigned sll to_integer(B_unsigned(4 downto 0));
    when "0101" =>  -- LSR
      result_unsigned := A_unsigned srl to_integer(B_unsigned(4 downto 0));
    when others =>  -- Default
      result_unsigned := (others => '0');
  end case;
  -- Convert result back to std_logic_vector
  Result <= std_logic_vector(result_unsigned);
  -- Set Zero flag using if-then-else for older VHDL versions
  if result_unsigned = 0 then
```

```
        Zero <= '1';
      else
        Zero <= '0';
      end if;
   end process;
end Behavioral;
```

## Control Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity ControlUnit is
   Port ( opcode : in STD_LOGIC_VECTOR(10 downto 0);
        reg_write : out STD_LOGIC;
        mem_write : out STD_LOGIC;
        mem_read : out STD_LOGIC;
        ALUSrc : out STD_LOGIC;
        ALUOp : out STD_LOGIC_VECTOR(3 downto 0));
end ControlUnit;
architecture Behavioral of ControlUnit is
begin
   process(opcode)
   begin
      case opcode is
        when "10001011000" => -- ADD
           reg_write <= '1'; mem_write <= '0'; mem_read <= '0'; ALUSrc <= '0'; ALUOp <= "0000";
        when "11001011000" => -- SUB
           reg_write <= '1'; mem_write <= '0'; mem_read <= '0'; ALUSrc <= '0'; ALUOp <= "0001";
```

```
        when "11111000010" => -- LDUR
          reg_write <= '1'; mem_write <= '0'; mem_read <= '1'; ALUSrc <= '1'; ALUOp <= "0000";
        when "11111000000" => -- STUR
          reg_write <= '0'; mem_write <= '1'; mem_read <= '0'; ALUSrc <= '1'; ALUOp <= "0000";
        when others =>
          reg_write <= '0'; mem_write <= '0'; mem_read <= '0'; ALUSrc <= '0'; ALUOp <= "0000";
      end case;
  end process;
end Behavioral;
```

# Instruction Memory

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity InstructionMemory is
  Port ( address : in STD_LOGIC_VECTOR(31 downto 0);
       instruction : out STD_LOGIC_VECTOR(31 downto 0));
end InstructionMemory;
architecture Behavioral of InstructionMemory is
  type memory_type is array (0 to 255) of STD_LOGIC_VECTOR(31 downto 0);
  signal memory : memory_type := (
    0 => "11111000010010100111100000001010", -- Example LDUR instruction
    1 => "10001011000010010101010000000101"  -- Example ADD instruction
    -- Add more instructions as needed
  );
begin
  instruction <= memory(to_integer(unsigned(address(7 downto 0))));
end Behavioral;
```

# PC

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
entity PC is
  Port ( clk : in STD_LOGIC;
       reset : in STD_LOGIC;
       PC_in : in STD_LOGIC_VECTOR(31 downto 0);
       PC_out : out STD_LOGIC_VECTOR(31 downto 0));
end PC;
architecture Behavioral of PC is
  signal PC_reg : STD_LOGIC_VECTOR(31 downto 0);
begin
  process(clk, reset)
  begin
    if reset = '1' then
      PC_reg <= (others => '0');
    elsif rising_edge(clk) then
      PC_reg <= PC_in;
    end if;
  end process;
  PC_out <= PC_reg;
end Behavioral;
```

# Register File

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity RegisterFile is
  Port ( clk : in STD_LOGIC;
       reset : in STD_LOGIC;
       read_reg1 : in STD_LOGIC_VECTOR(4 downto 0);
       read_reg2 : in STD_LOGIC_VECTOR(4 downto 0);
       write_reg : in STD_LOGIC_VECTOR(4 downto 0);
       write_data : in STD_LOGIC_VECTOR(31 downto 0);
       reg_write : in STD_LOGIC;
       read_data1 : out STD_LOGIC_VECTOR(31 downto 0);
       read_data2 : out STD_LOGIC_VECTOR(31 downto 0));
end RegisterFile;
architecture Behavioral of RegisterFile is
  type reg_array is array (0 to 31) of STD_LOGIC_VECTOR(31 downto 0);
  signal registers : reg_array := (others => (others => '0'));
begin
  read_data1 <= registers(to_integer(unsigned(read_reg1)));
  read_data2 <= registers(to_integer(unsigned(read_reg2)));
process(clk)
begin
  if rising_edge(clk) then
    if reg_write = '1' then
      registers(to_integer(unsigned(write_reg))) <= write_data;
    end if;
  end if;
end process;
```

end Behavioral;

# Constraints

## Clock Source

set_property -dict {PACKAGE_PIN <new_pin> IOSTANDARD LVCMOS33} [get_ports clk]

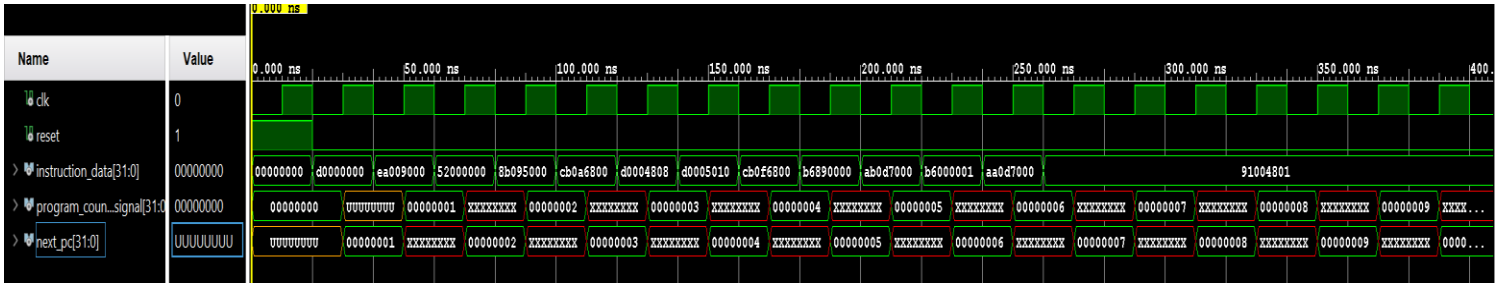
create_clock -period 10.000 -name clk [get_ports clk]

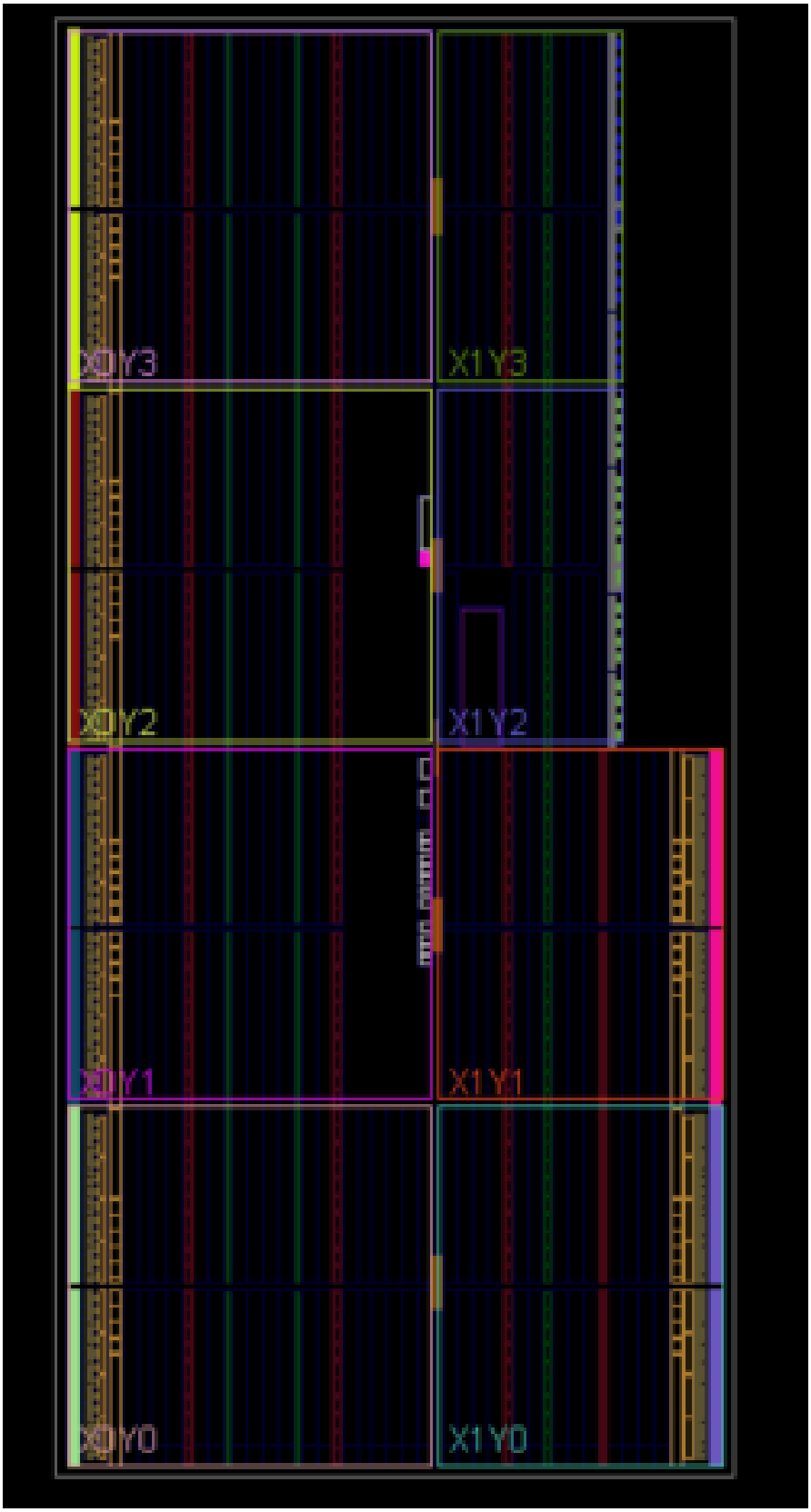
## Reset Pin

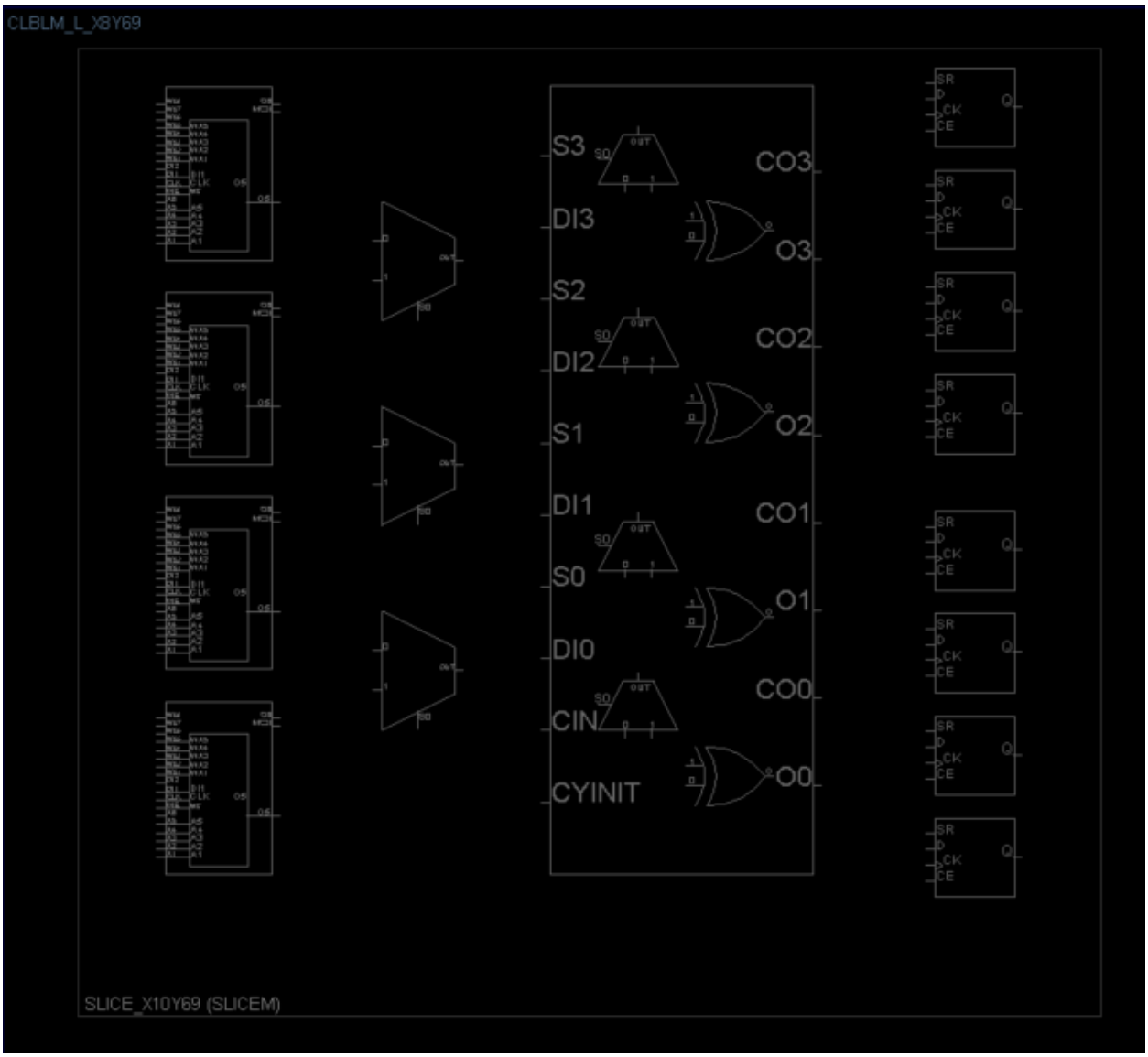
set_property -dict {PACKAGE_PIN V4 IOSTANDARD LVCMOS33} [get_ports reset]

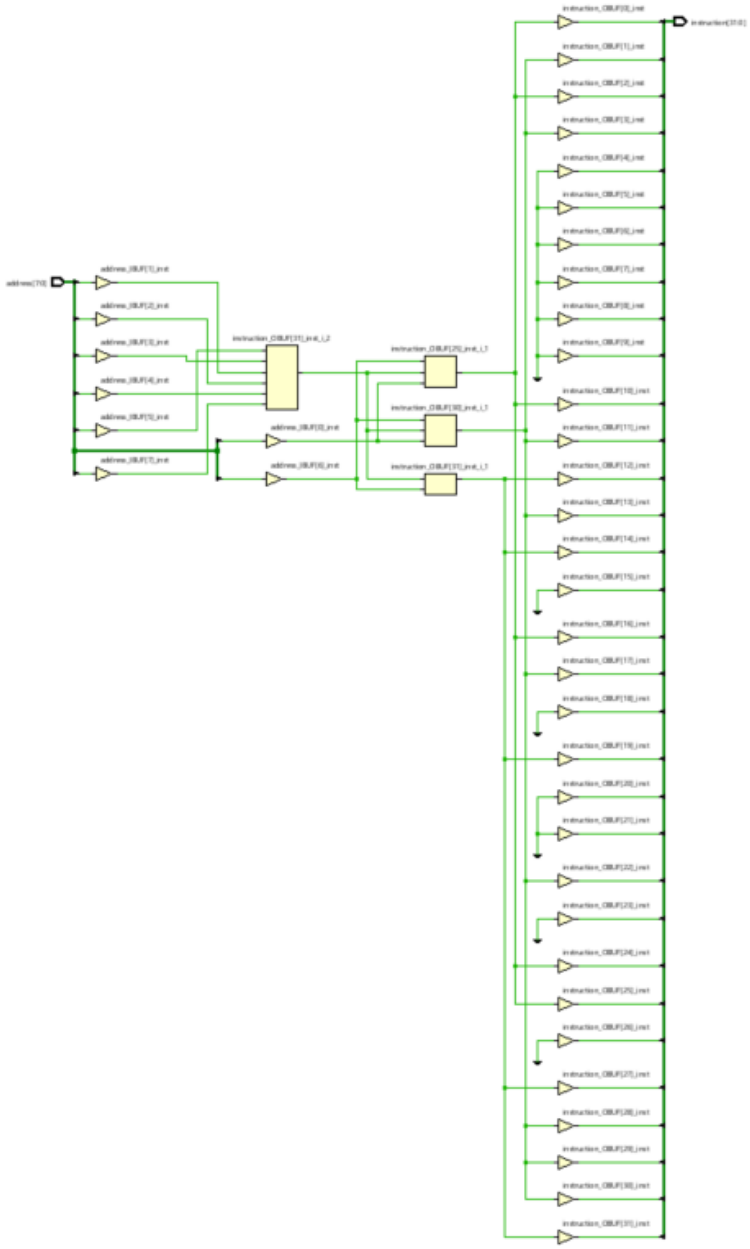
## Simulation results

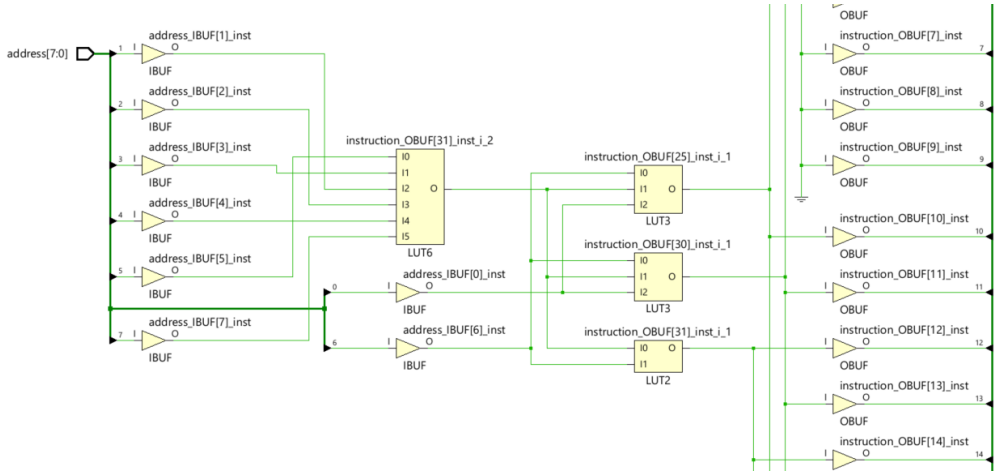
The following is the results of the data path and code described previously, also included are the data paths for the first part of the assignment.  These were used to test the functionality of the processor and it was being constructed to ensure that the desired values were being produced. Below is the out put of the processor, the photos that follow are the data paths used to construct the processor.

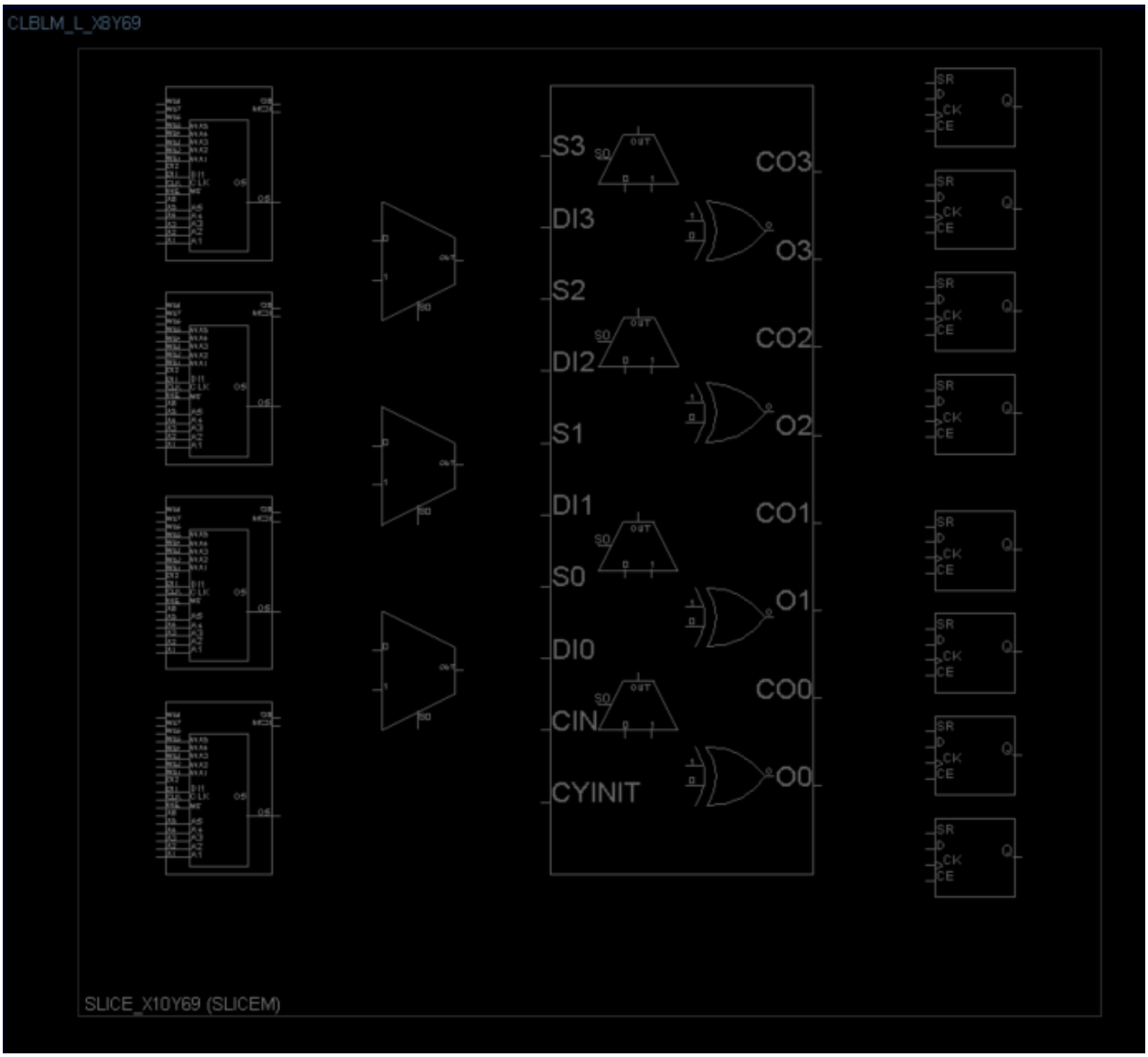


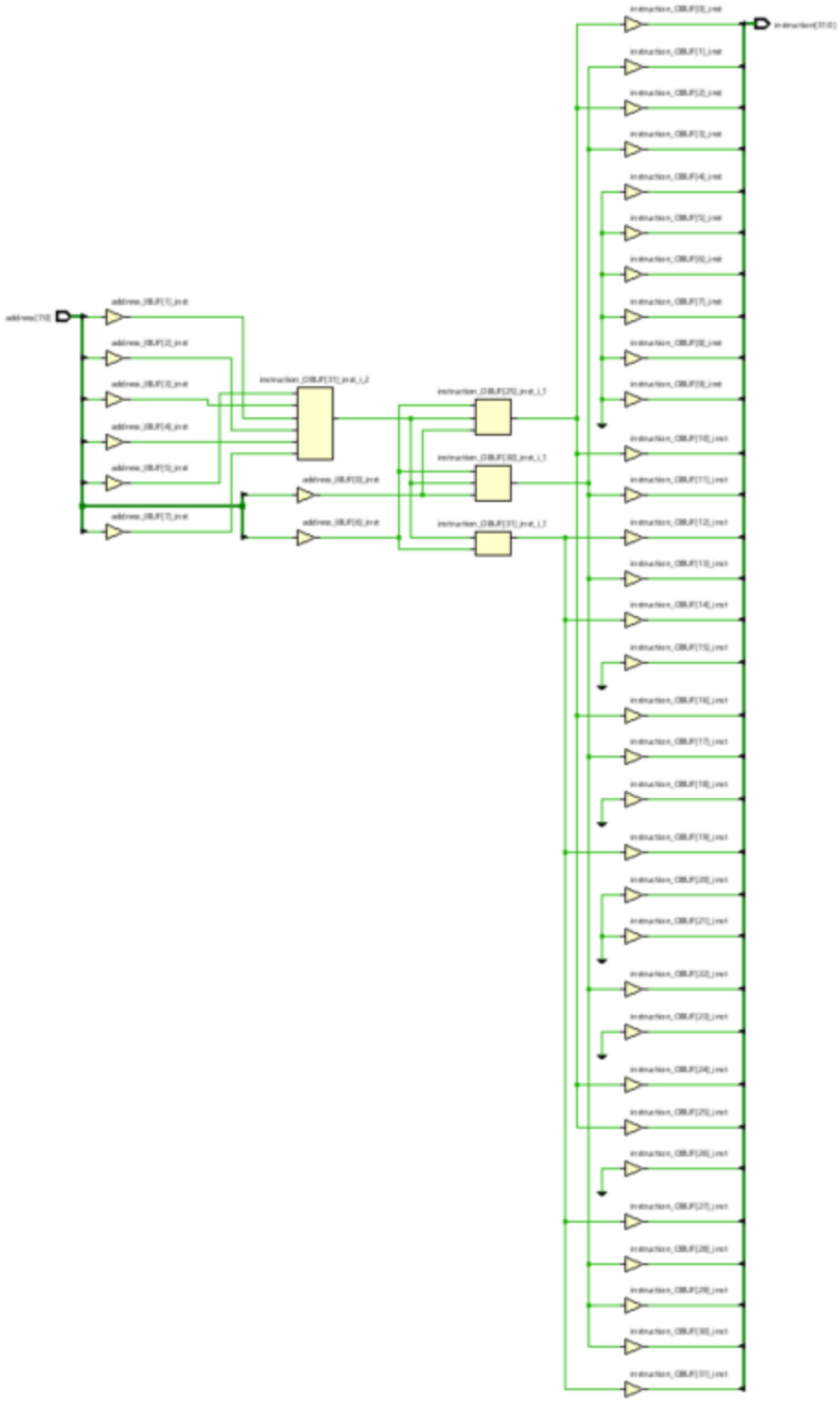

X0Y3
X1Y3
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0

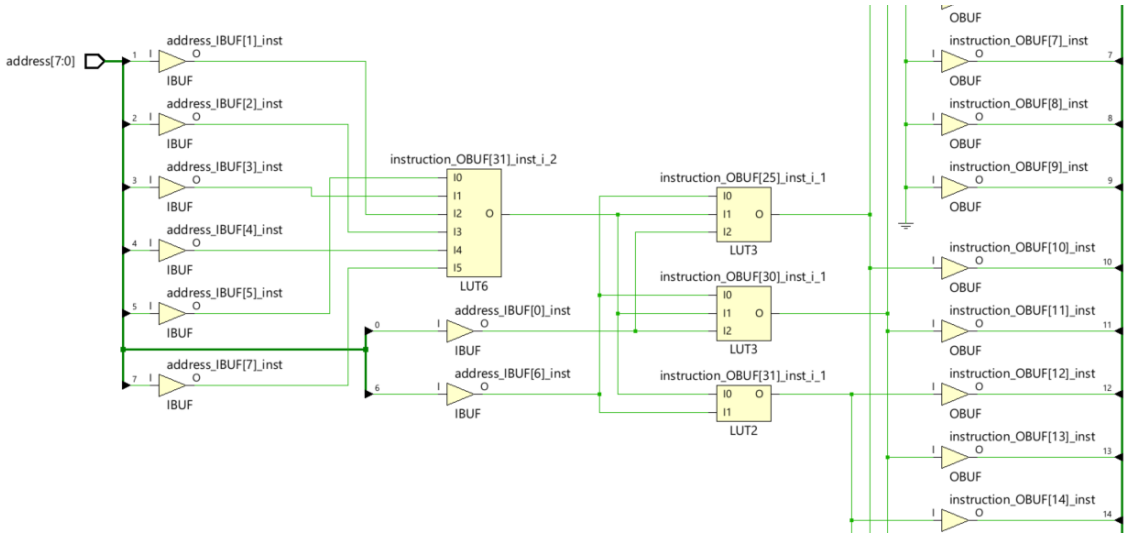

CLBLM_L_X8Y69
S3
DI3
S2
DI2
S1
DI1
S0
DI0
CIN
CYINIT
CO3
O3
CO2
O2
CO1
O1
CO0
O0
SR
D
CK
CE
Q
SLICE_X10Y69 (SLICEM)

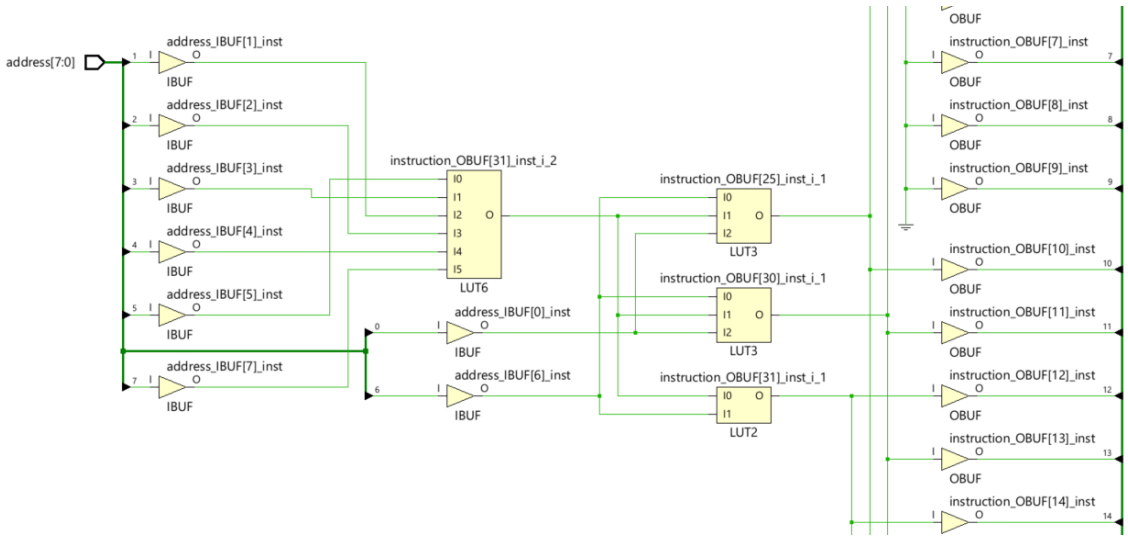



address[7:0]
address_IBUF[1]_inst
IBUF
address_IBUF[2]_inst
IBUF
address_IBUF[3]_inst
IBUF
address_IBUF[4]_inst
IBUF
address_IBUF[5]_inst
IBUF
address_IBUF[7]_inst
IBUF
instruction_OBUF[31]_inst_i_2
I0
I1
I2
I3
I4
I5
O
LUT6
address_IBUF[0]_inst
IBUF
address_IBUF[6]_inst
IBUF
instruction_OBUF[25]_inst_i_1
I0
I1
I2
O
LUT3
instruction_OBUF[30]_inst_i_1
I0
I1
I2
O
LUT3
instruction_OBUF[31]_inst_i_1
I0
I1
O
LUT2
OBUF
instruction_OBUF[7]_inst
OBUF
instruction_OBUF[8]_inst
OBUF
instruction_OBUF[9]_inst
OBUF
instruction_OBUF[10]_inst
OBUF
instruction_OBUF[11]_inst
OBUF
instruction_OBUF[12]_inst
OBUF
instruction_OBUF[13]_inst
OBUF
instruction_OBUF[14]_inst


CLBLM_L_X8Y69
S3
DI3
S2
DI2
S1
DI1
S0
DI0
CIN
CYINIT
CO3
O3
CO2
O2
CO1
O1
CO0
O0
SR
D
CK
CE
Q
SLICE_X10Y69 (SLICEM)




address[7:0]
address_IBUF[1]_inst
IBUF
address_IBUF[2]_inst
IBUF
address_IBUF[3]_inst
IBUF
address_IBUF[4]_inst
IBUF
address_IBUF[5]_inst
IBUF
address_IBUF[7]_inst
IBUF
address_IBUF[0]_inst
IBUF
address_IBUF[6]_inst
IBUF
instruction_OBUF[31]_inst_i_2
LUT6
instruction_OBUF[25]_inst_i_1
LUT3
instruction_OBUF[30]_inst_i_1
LUT3
instruction_OBUF[31]_inst_i_1
LUT2
OBUF
instruction_OBUF[7]_inst
OBUF
instruction_OBUF[8]_inst
OBUF
instruction_OBUF[9]_inst
OBUF
instruction_OBUF[10]_inst
OBUF
instruction_OBUF[11]_inst
OBUF
instruction_OBUF[12]_inst
OBUF
instruction_OBUF[13]_inst
OBUF
instruction_OBUF[14]_inst


address[7:0]
address_IBUF[1]_inst
IBUF
address_IBUF[2]_inst
IBUF
address_IBUF[3]_inst
IBUF
address_IBUF[4]_inst
IBUF
address_IBUF[5]_inst
IBUF
address_IBUF[7]_inst
IBUF
address_IBUF[0]_inst
IBUF
address_IBUF[6]_inst
IBUF
instruction_OBUF[31]_inst_i_2
LUT6
instruction_OBUF[25]_inst_i_1
LUT3
instruction_OBUF[30]_inst_i_1
LUT3
instruction_OBUF[31]_inst_i_1
LUT2
OBUF
instruction_OBUF[7]_inst
OBUF
instruction_OBUF[8]_inst
OBUF
instruction_OBUF[9]_inst
OBUF
instruction_OBUF[10]_inst
OBUF
instruction_OBUF[11]_inst
OBUF
instruction_OBUF[12]_inst
OBUF
instruction_OBUF[13]_inst
OBUF
instruction_OBUF[14]_inst